ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas

Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE LABORATORIO · SEMANA 4
CLASIFICACIÓN DE VARIABLES Y POBLACIÓN VS. MUESTRA

Estadística y Programación para Ciencia de Datos I · IDSD422

Campo	Información
Estudiante	Andrea Nicol Cumbal Gordón
Usuario de GitHub	andrea0119C
Docente	Christian Suárez
Período académico	2026-B
Fecha	23/10/2026
Repositorio (fork)	https://github.com/andrea0119C/IDSD422-lab-muestreo-poblacion.git

Informe elaborado a partir de los resultados de clasificación y muestreo obtenidos en Python y R con los archivos clientes.csv y clientes_400.csv.

1. Introducción

RetailAndina S. A. necesita conocer las características de su base de fidelización y estimar el ingreso mensual promedio de sus clientes. En esta práctica trabajé con una base de 12 registros para clasificar variables y con una población de 400 clientes para comparar muestras de tamaño 20. Verifiqué los tipos de datos en Python y R y contrasté tres diseños de selección. Esta revisión es importante porque una variable mal clasificada puede llevar a un análisis incorrecto y porque aumentar el tamaño de una muestra no elimina el sesgo de selección.

2. Objetivos

Objetivo general:

Clasificar las variables de fidelización y evaluar tres diseños de muestreo para estimar el ingreso mensual promedio de los clientes de RetailAndina S. A.

Objetivos específicos:

Identificar y justificar los tipos estadísticos de las variables.

Comprobar tipos inferidos, valores únicos y categorías ordenadas en Python y R.

Implementar muestreos por conveniencia, aleatorio simple y estratificado.

Comparar las estimaciones con la media poblacional e interpretar el sesgo y la variabilidad.

3. Desarrollo de la práctica

3.1. Parte A: clasificación de variables

Leí data/clientes.csv y confirmé 12 clientes y ocho columnas (siete variables y un identificador). Utilicé dtypes y unique() en Python, y str() y unique() en R. La clasificación se realizó según el significado estadístico de cada variable, no únicamente por el tipo inferido por el lenguaje.

Variable	Tipo estadístico	Justificación y verificación
cliente_id	Identificador nominal	Código sin magnitud. Python str / R chr; 12 únicos.
edad	Numérica discreta	Años cumplidos registrados como enteros. int64 / int; 12 únicos.
numero_hijos	Numérica discreta	Conteo entero. int64 / int; 5 valores (0–4).
genero	Categórica nominal	Sin orden. str / chr; 3 categorías: Femenino, Masculino, Otro.
nivel_educacion	Categórica ordinal	Cuatro niveles con jerarquía; str / chr antes de ordenar.
ciudad	Categórica nominal	Sin orden. str / chr; Quito, Guayaquil y Cuenca.
ingresos_mensuales	Numérica continua	Magnitud monetaria, aunque registrada como int64 / int; 12 únicos.
satisfaccion	Categórica ordinal	Escala ordenada, no cantidad proporcional; int64 / int; 4 únicos (2–5).

*La edad se clasificó como discreta por su registro en años cumplidos; conceptualmente el tiempo transcurrido desde el nacimiento puede medirse de forma continua.

3.2. Orden de las categorías y control de calidad

Declaré nivel_educacion como categoría ordenada con la secuencia Primaria < Secundaria < Universidad < Posgrado. La función min() devolvió Primaria. Para satisfacción declaré los niveles 1 < 2 < 3 < 4 < 5 y observé que el mínimo en los datos fue 2, pues no existían respuestas de nivel 1. En una prueba de calidad escribí Postgrado en lugar de Posgrado: la transformación generó dos valores faltantes. Esta incidencia no correspondía a datos originales vacíos, sino a una etiqueta incorrecta.

P1. El dtype de satisfaccion fue int64 en Python e int en R. Esto no basta para clasificarla, porque los números indican categorías ordenadas y no una magnitud con relaciones proporcionales: una puntuación de 4 no significa el doble de satisfacción que una de 2. Si no se declara el orden, una variable textual podría ordenarse alfabéticamente y producir comparaciones sin sentido.

3.3. Parte B: población y parámetro

Leí data/clientes_400.csv y confirmé 400 clientes y ocho columnas. La variable estudiada fue ingresos_mensuales. La población se distribuyó en Quito (200), Guayaquil (120) y Cuenca (80). La media de los 400 registros, calculada con mean(), fue 1.330,72 USD.

Concepto	**Resultado**
Población	400 clientes
Variable de interés	ingresos_mensuales
Parámetro poblacional	1.330,72 USD
Composición de la población	Quito 50 %; Guayaquil 30 %; Cuenca 20 %

Aunque conocí el parámetro de esta base pequeña, realicé los muestreos para observar sus errores frente al valor real. En aplicaciones con poblaciones grandes, disponer de una muestra puede reducir costos y tiempo; sin embargo, su utilidad depende de cómo se seleccionen los registros.

3.4. Implementación de los diseños

Seleccioné los primeros 20 registros con head() para la muestra por conveniencia; sorteé 20 clientes sin reemplazo mediante sample() para la aleatoria simple; y seleccioné el 5 % dentro de cada ciudad con groupby() / slice_sample() para la estratificada. Utilicé semillas explícitas para permitir repetir los sorteos. Presento los resultados de la ejecución en R, sin mezclar sus semillas con las de Python.

Diseño (n=20)	**Media (USD)**	**Ciudad de los 20 seleccionados**	**Diferencia (USD)**
Conveniencia	1.348,50	Quito 20; Guayaquil 0; Cuenca 0	+17,78
Aleatorio simple	1.336,00	Quito 8; Guayaquil 4; Cuenca 8	+5,28
Estratificado	1.287,00	Quito 10; Guayaquil 6; Cuenca 4	−43,22

3.5. Variabilidad, sesgo e interpretación

Para observar la variabilidad repetí el muestreo aleatorio simple de 20 clientes con cinco semillas en R. Aun manteniendo el mismo tamaño, las medias cambiaron porque cada selección incluyó clientes diferentes.

Semilla	**1**	**2**	**3**	**4**	**5**
Media (USD)	1.336,00	1.405,50	1.235,00	1.491,50	1.288,00

P2. El parámetro poblacional fue de 1.330,72 USD. Las medias estimadas mediante conveniencia, aleatorio simple y estratificado fueron de 1.348,50; 1.318,00 y 1.226,00 USD, respectivamente. En esta ejecución, el muestreo aleatorio simple presentó la menor diferencia absoluta respecto al parámetro (12,72 USD). Sin embargo, esto no significa que sea el mejor diseño en todas las situaciones, ya que una sola ejecución no permite evaluar su comportamiento general y la cercanía también puede depender del azar.

P3. En el muestreo por conveniencia, los 20 clientes seleccionados pertenecían a Quito, aunque esta ciudad representa únicamente el 50 % de la población. La ausencia de Guayaquil y Cuenca evidenció un problema de cobertura. Aunque la diferencia respecto al parámetro fue de apenas +17,78 USD, esto no garantiza que la muestra sea representativa. Aumentar el número de registros tampoco corrige automáticamente el sesgo si se mantiene el mismo procedimiento de selección.

P4. Al repetir el muestreo aleatorio simple con cinco semillas en R, obtuve medias entre 1.233,00 y 1.385,50 USD, con una diferencia de 152,50 USD entre el valor máximo y mínimo. Esto me permitió observar que las estimaciones pueden cambiar según los clientes seleccionados, incluso cuando todas las muestras tienen 20 registros. Por ello, recomendaría a la empresa considerar la variabilidad muestral y utilizar medidas de incertidumbre antes de tomar decisiones basadas en una sola estimación.

P5. Utilizaría el muestreo por conveniencia para realizar pruebas piloto sin buscar conclusiones sobre toda la población. El muestreo aleatorio simple sería adecuado para estimaciones generales cuando se dispone de una lista completa de clientes. Finalmente, utilizaría el estratificado cuando sea necesario garantizar la representación proporcional de Quito, Guayaquil y Cuenca. En este ejercicio, se seleccionaron 10, 6 y 4 clientes, respectivamente, aunque la media obtenida fue la más alejada del parámetro. Esto demuestra que mantener las proporciones geográficas no elimina la variabilidad muestral.

3.6. Procedimiento técnico e incidencias

Trabajé con los cuatro scripts del repositorio, dos de clasificación y dos de muestreo, en Python y R. Al preparar los archivos, corregí referencias a rutas relativas para ejecutar desde la raíz del proyecto. En R instalé dplyr para usar slice_sample(); antes de instalarlo, el script mostraba un error de paquete ausente. También corregí una relectura innecesaria del CSV durante la prueba de Postgrado. Conservé las salidas y el código como evidencias de los resultados descritos.

4. ¿Qué aprendí con esta práctica?

Comprendí que la clasificación estadística depende del significado de los valores y no solo del tipo reconocido por el programa. La variable satisfacción fue el ejemplo más claro: aparecía como entera, pero debía tratarse como ordinal. Al declarar los niveles de educación también observé la importancia de respetar el orden lógico y de escribir las etiquetas exactamente: el cambio de Posgrado por Postgrado produjo dos faltantes.

En muestreo confirmé que no basta con seleccionar veinte filas y calcular una media. La muestra por conveniencia contenía exclusivamente registros de Quito y dejó fuera dos ciudades. También observé que los sorteos aleatorios, incluso con igual tamaño, pueden producir estimaciones distintas. La parte más exigente fue corregir las rutas del proyecto y completar la ejecución de R con dplyr. Para la siguiente práctica comprobaría primero el directorio de trabajo, ejecutaría los scripts completos y guardaría cada evidencia al finalizarla.

5. Conclusiones

1. La clasificación de variables me permitió comprobar que el tipo de dato reconocido por Python y R no siempre corresponde a su clasificación estadística. En el archivo clientes.csv, analicé 12 registros y 8 columnas. Por ejemplo, satisfaccion fue identificada como un dato entero, pero la clasifiqué como categórica ordinal porque representa niveles ordenados. Además, al escribir Postgrado en lugar de Posgrado, se generaron 2 valores faltantes, lo que evidenció la importancia de verificar las categorías antes de realizar un análisis.

2. Al comparar los tres diseños de muestreo con el parámetro poblacional de 1.330,72 USD, observé que el aleatorio simple obtuvo la estimación más cercana, con una media de 1.318,00 USD y una diferencia absoluta de 12,72 USD. El muestreo por conveniencia registró 1.348,50 USD, mientras que el estratificado obtuvo 1.226,00 USD. Sin embargo, comprendí que una sola ejecución no permite determinar qué método es mejor, porque los resultados también dependen de los clientes seleccionados. Esto se evidenció al repetir cinco sorteos en R, cuyas medias variaron entre 1.233,00 y 1.385,50 USD.

3. Recomendaría a la gerencia utilizar métodos de muestreo probabilísticos cuando necesite obtener conclusiones sobre sus 400 clientes. La muestra por conveniencia incluyó únicamente 20 clientes de Quito, dejando fuera a Guayaquil y Cuenca, mientras que el muestreo estratificado aseguró una distribución proporcional de 10 clientes de Quito, 6 de Guayaquil y 4 de Cuenca. Aunque el estratificado presentó la mayor diferencia respecto al parámetro en esta ejecución, permitió mantener la representación de las tres ciudades. Por ello, considero que antes de tomar decisiones es fundamental evaluar el procedimiento de selección y la variabilidad de los resultados, ya que el tamaño de una muestra no garantiza por sí solo su representatividad.

6. Bibliografía

Bruce, P., Bruce, A., y Gedeck, P. (2020). Practical Statistics for Data Scientists (2.ª ed.). O'Reilly.

pandas development team. (s. f.). pandas.CategoricalDtype. https://pandas.pydata.org/docs/reference/api/pandas.CategoricalDtype.html

Wickham, H., François, R., Henry, L., Müller, K., y Vaughan, D. (s. f.). dplyr: A Grammar of Data Manipulation. https://dplyr.tidyverse.org/

RetailAndina S. A. (2026). clientes.csv y clientes_400.csv [Conjuntos de datos del laboratorio IDSD422, repositorio de la asignatura].

Declaración de uso de inteligencia artificial

Utilicé ChatGPT como herramienta de apoyo para resolver dudas puntuales sobre algunas funciones de Python y R, comprender mensajes de error y realizar pequeñas correcciones en el código durante la práctica. Los resultados fueron obtenidos mediante la ejecución de los programas y su respectiva verificación.

Repositorio: https://github.com/andrea0119C/IDSD422-lab-muestreo-poblacion.git

Anexo A. Evidencias de ejecución

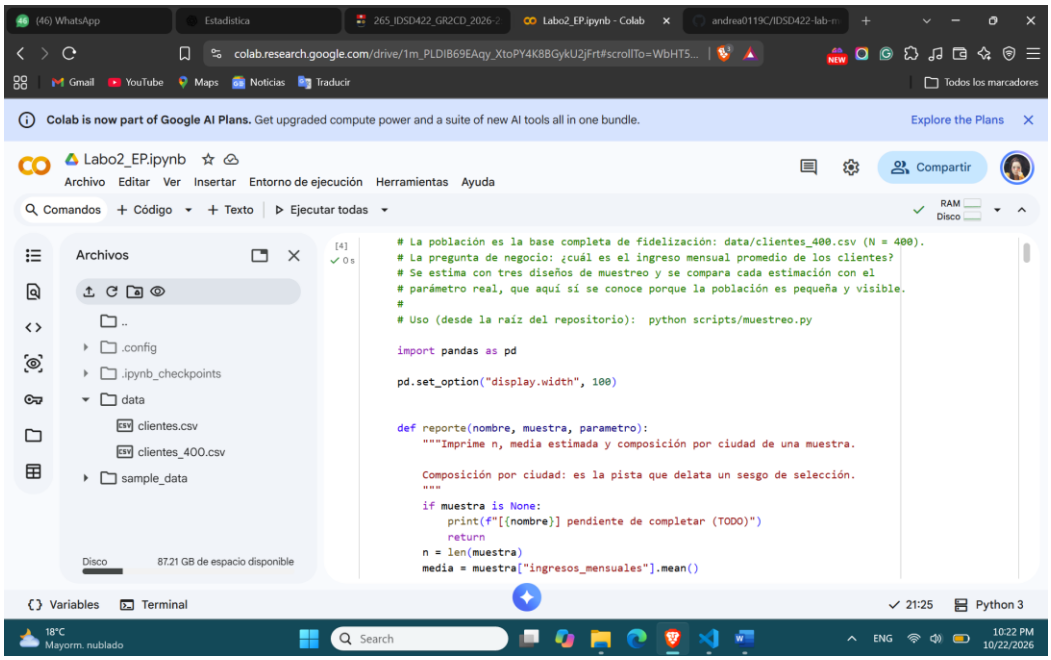


Figura 1. Inicio del ejercicio de muestreo en Google Colab: archivos de la base de clientes y función de reporte.

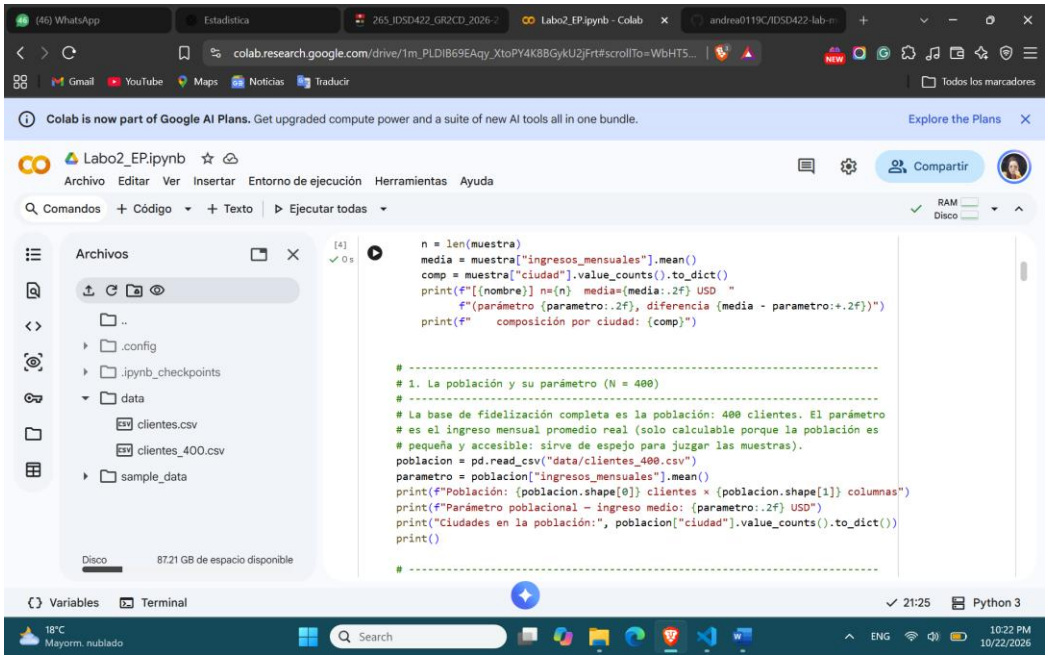


Figura 2. Preparación de la carga de la población y cálculo del parámetro en Google Colab.

Anexo B. Evidencias de ejecución

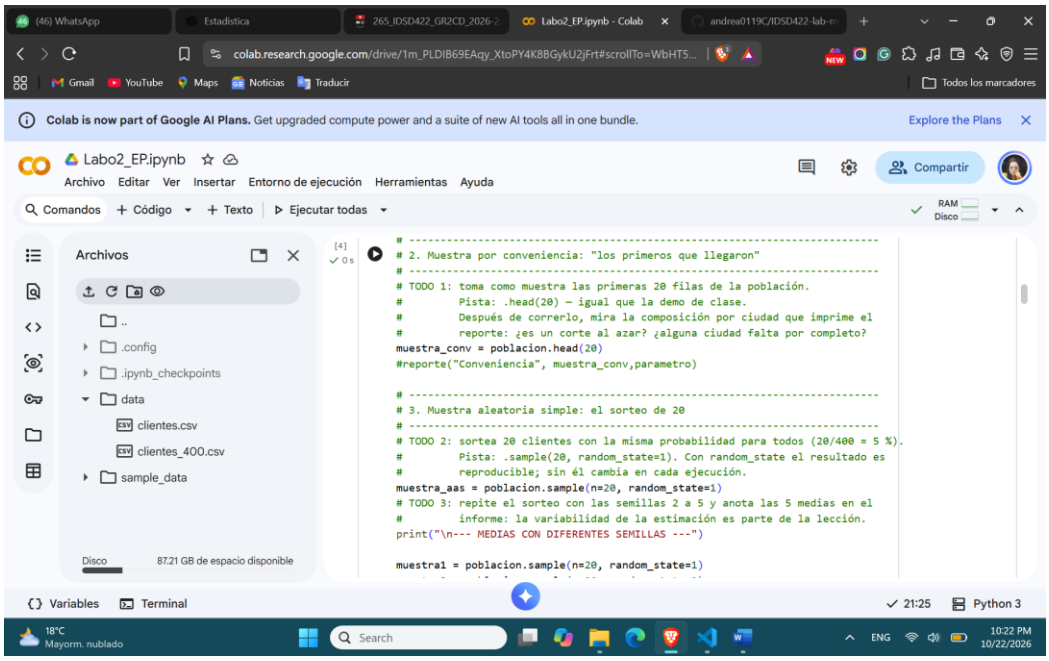


Figura 3. Implementación de la selección por conveniencia y aleatoria simple en Python.

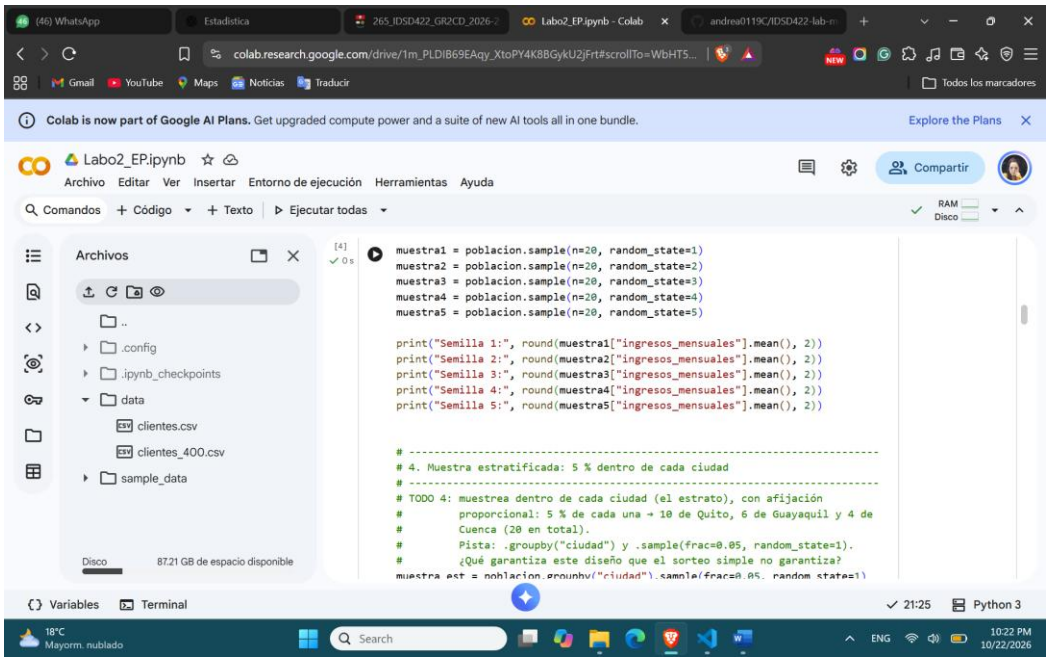


Figura 4. Implementación de las cinco semillas y preparación del muestreo estratificado en Python.

Anexo C. Evidencias de ejecución

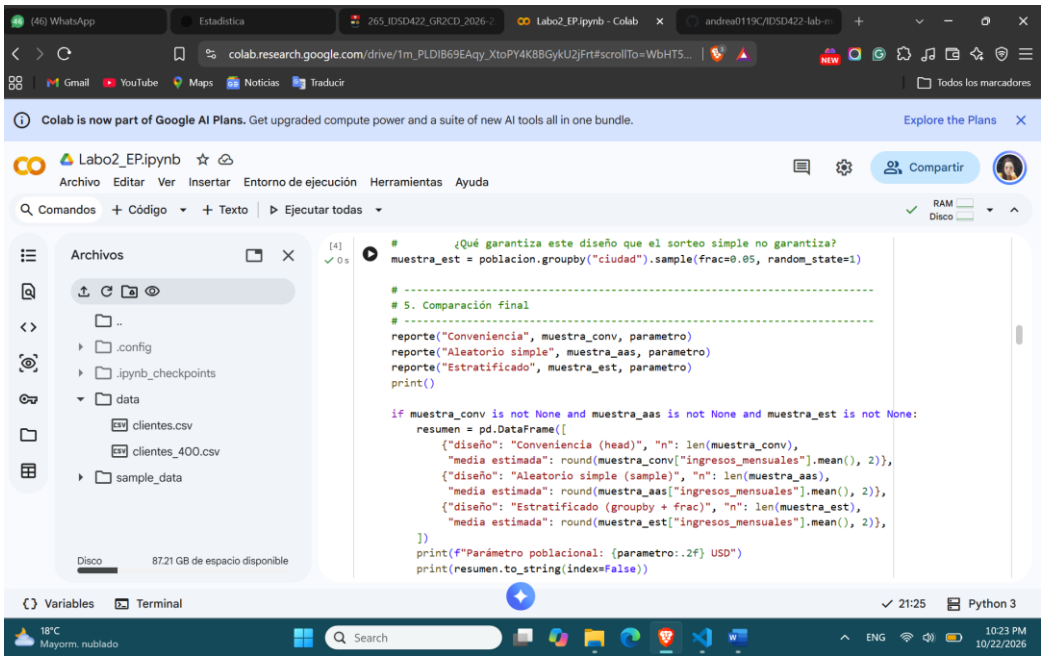


Figura 5. Muestreo estratificado y construcción de la tabla de comparación en Python.

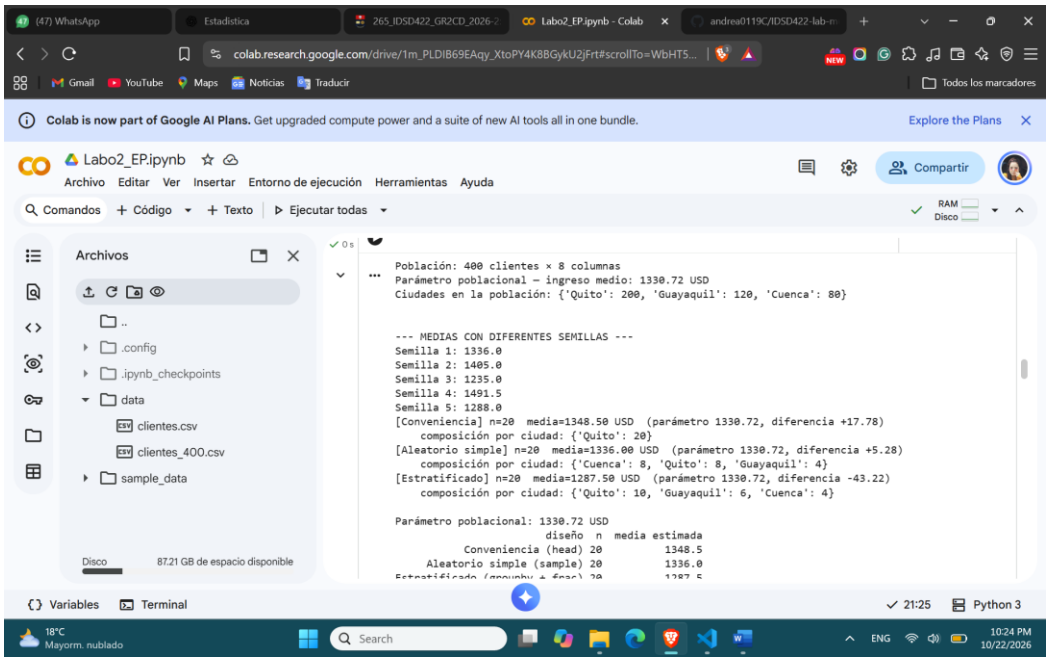


Figura 6. Salida de muestreo en Python: parámetro poblacional, estimaciones y composición por ciudad.

Anexo D. Evidencias de ejecución

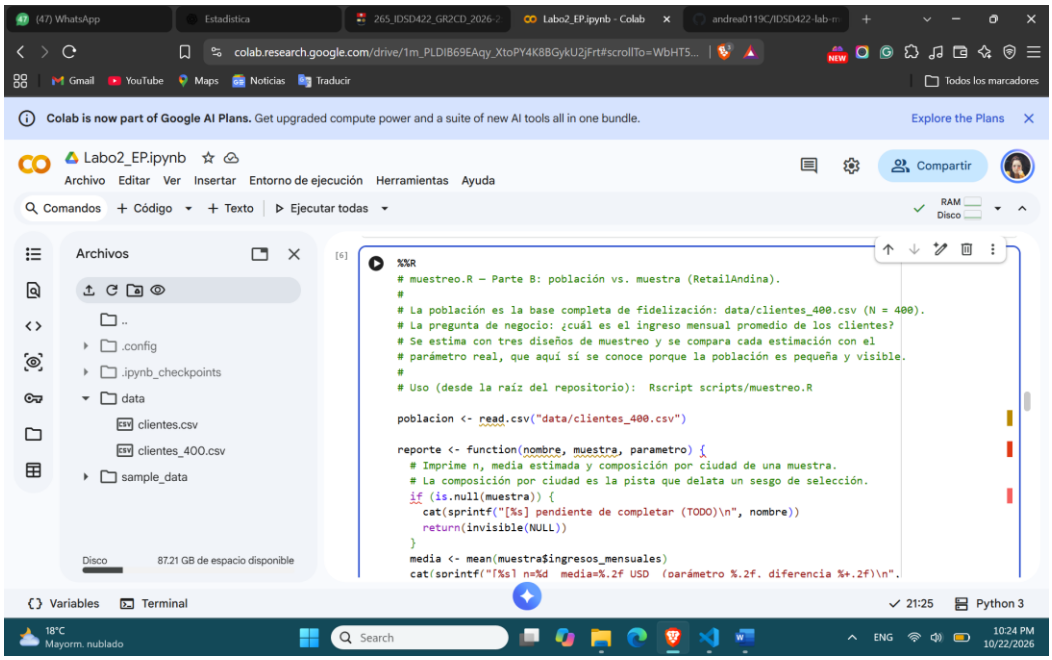


Figura 7. Implementación de la población y la función de reporte en R desde Colab.

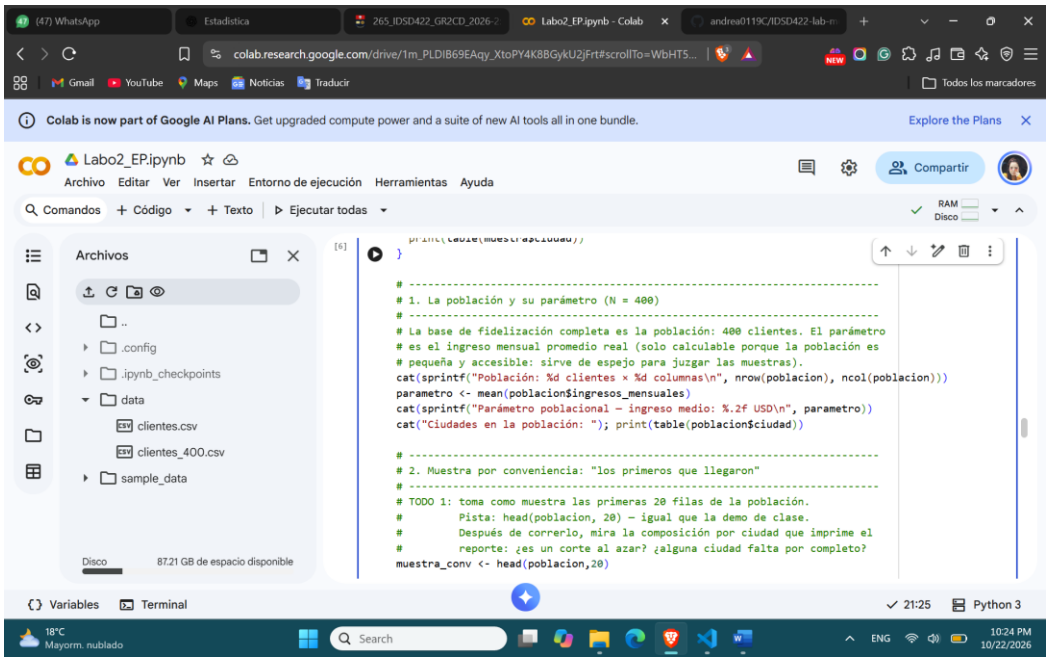


Figura 8. Muestra por conveniencia y cálculo de la media poblacional en R.

Anexo E. Evidencias de ejecución

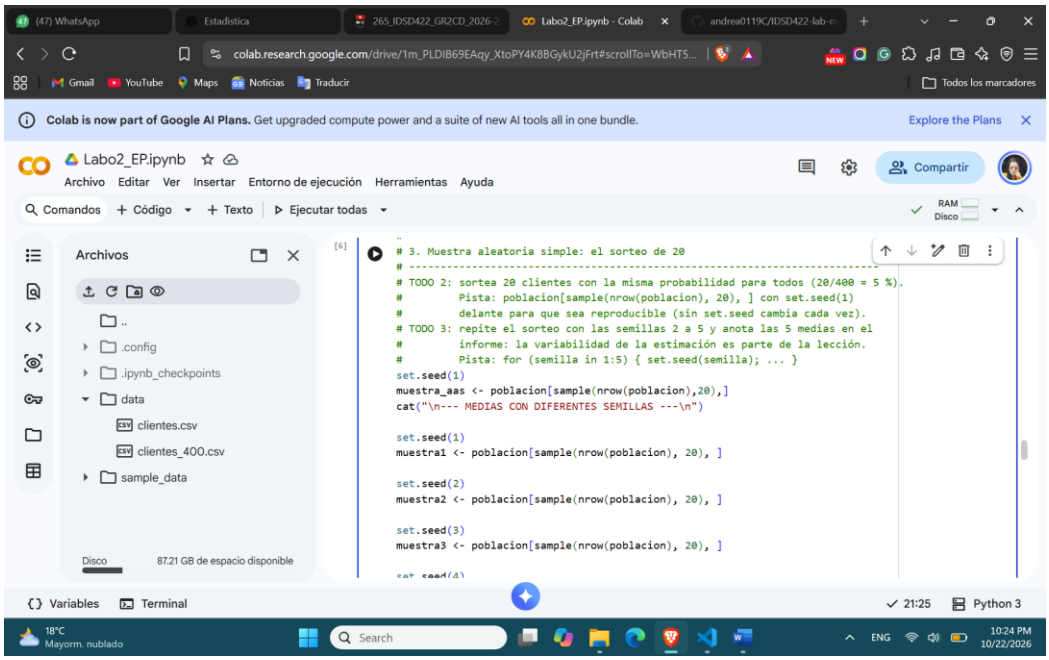


Figura 9. Código de selección aleatoria con semillas individuales en R.

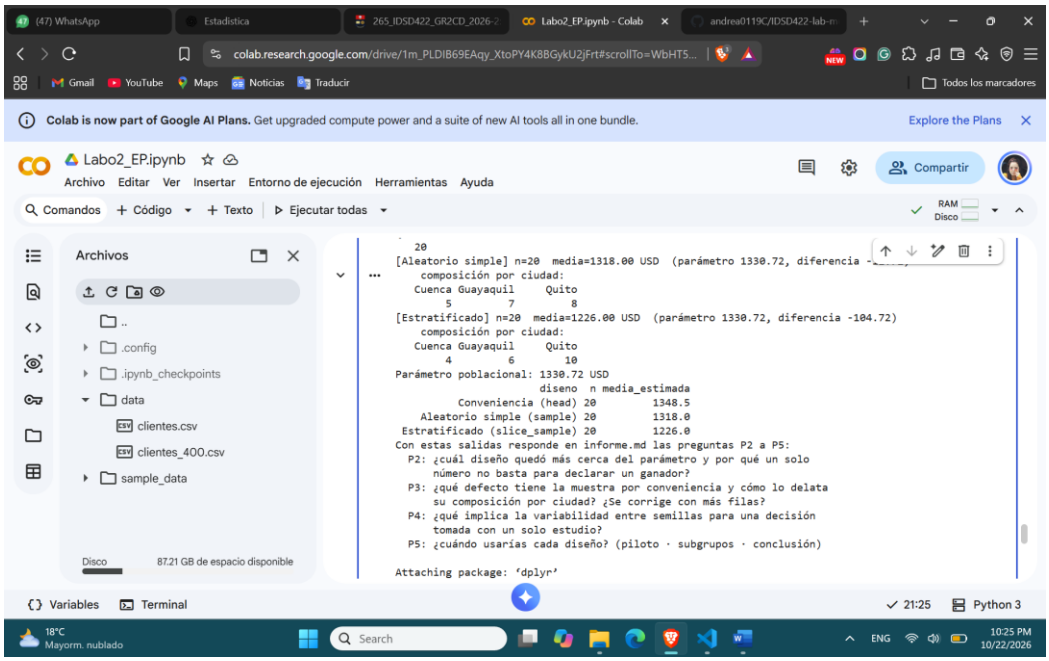


Figura 10. Resultados del muestreo en R y diferencias respecto del parámetro.

Anexo F. Evidencias de ejecución

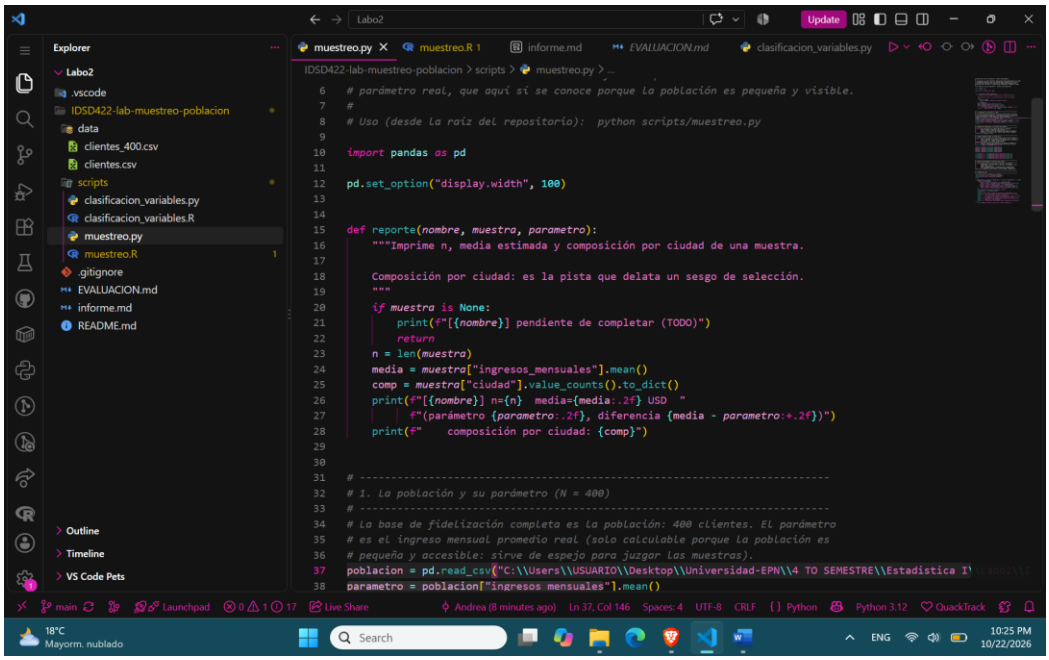


Figura 11. Archivo muestreo.py en VS Code: código y estructura del repositorio.

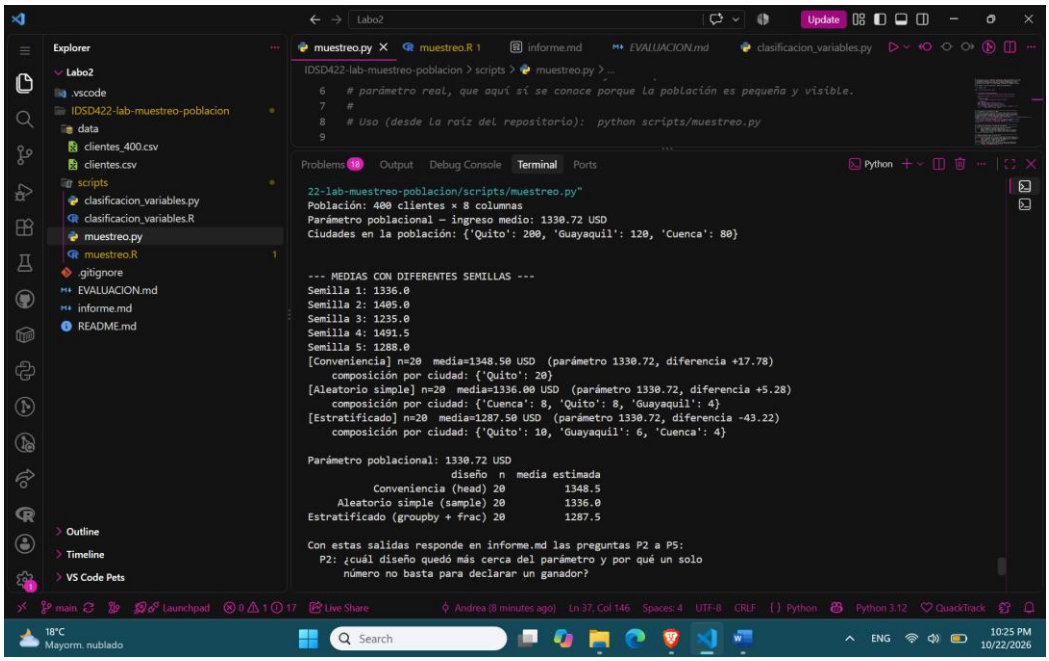


Figura 12. Ejecución local de muestreo.py: medias muestrales y tabla comparativa.

Anexo G. Evidencias de ejecución

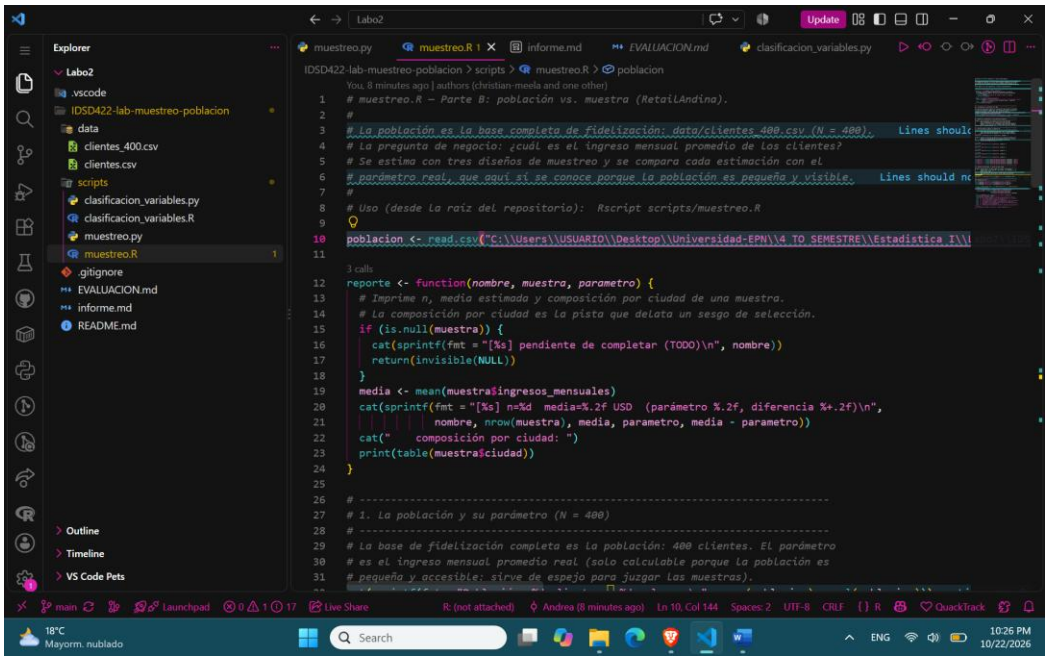


Figura 13. Archivo muestreo.R en VS Code: función de reporte y parámetros.

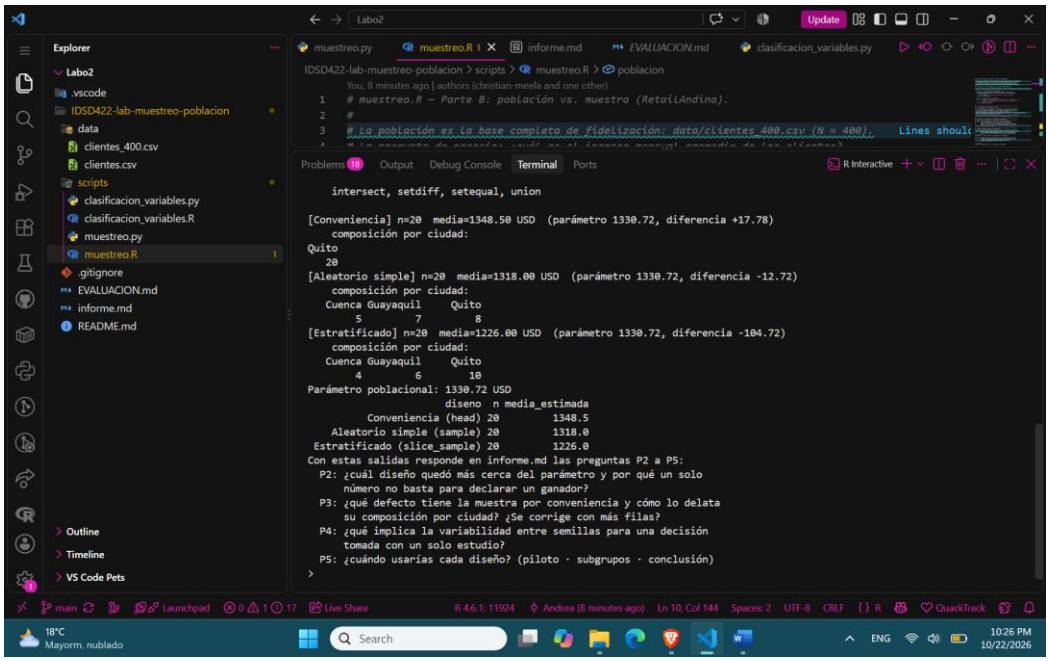


Figura 14. Ejecución de muestreo.R: composición de las tres muestras y comparaciones.

Anexo H. Evidencias de ejecución

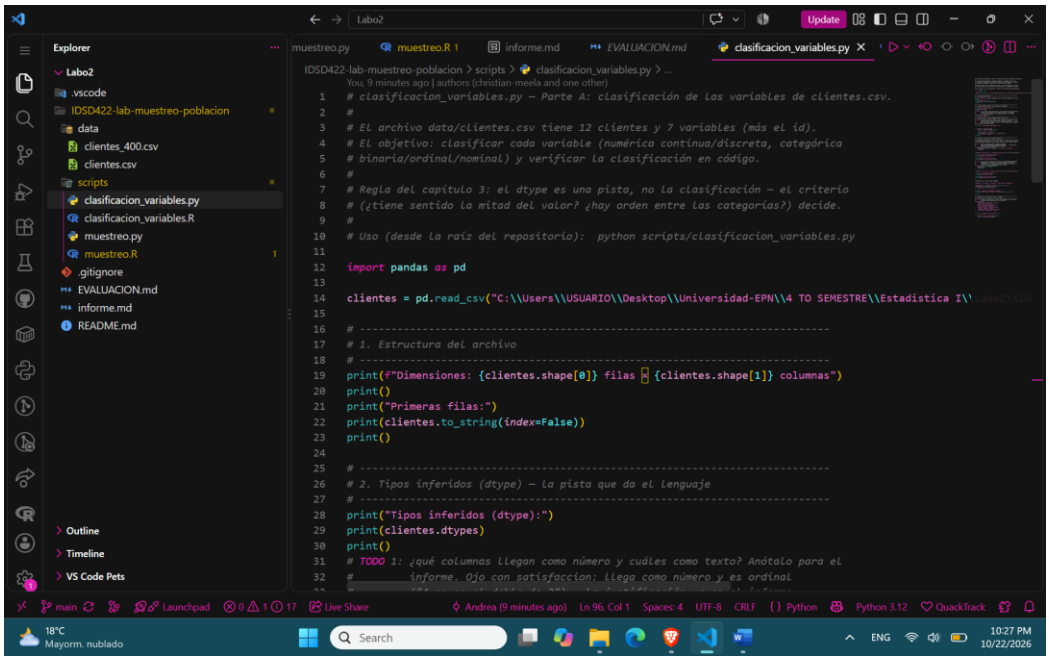


Figura 15. Código de clasificacion_variables.py en VS Code.

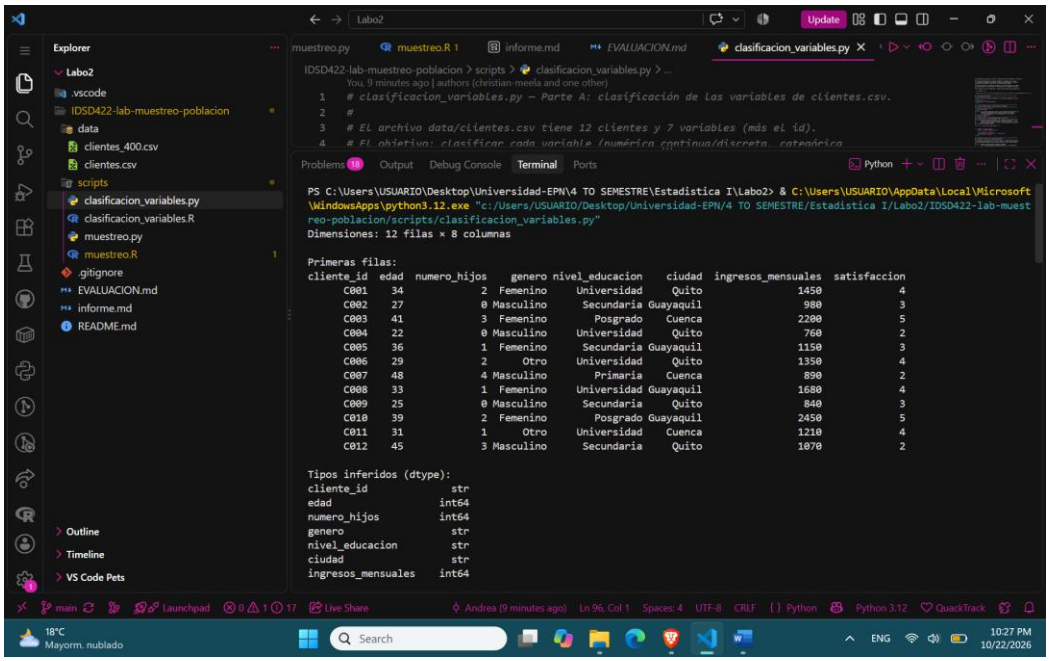


Figura 16. Lectura de clientes.csv en Python: 12 observaciones y tipos inferidos.

Anexo I. Evidencias de ejecución

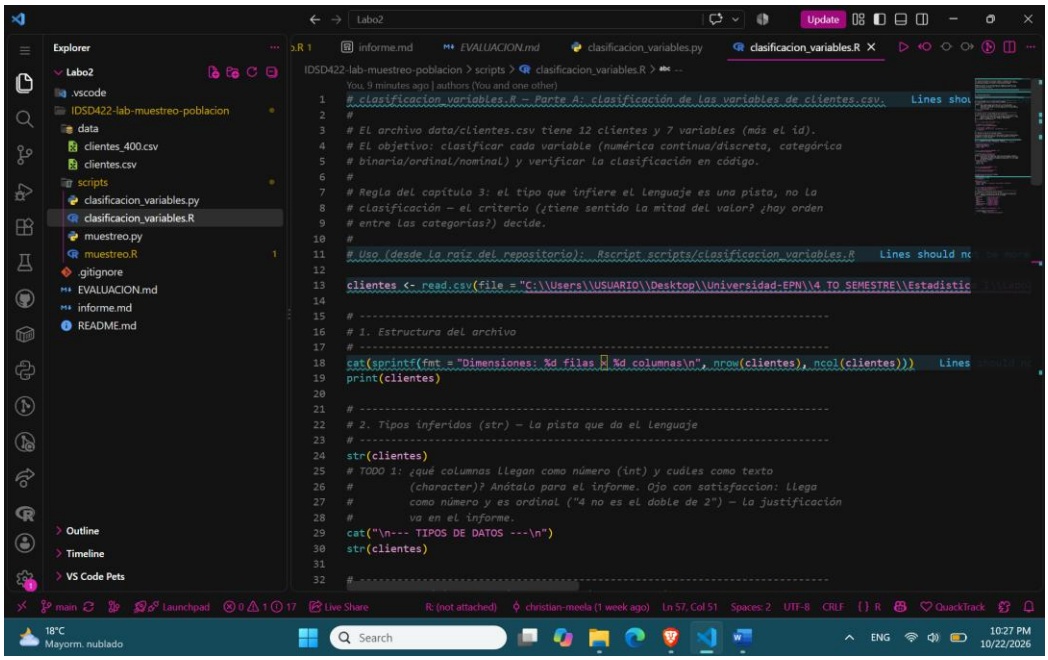


Figura 17. Código de clasificacion_variables.R en VS Code.

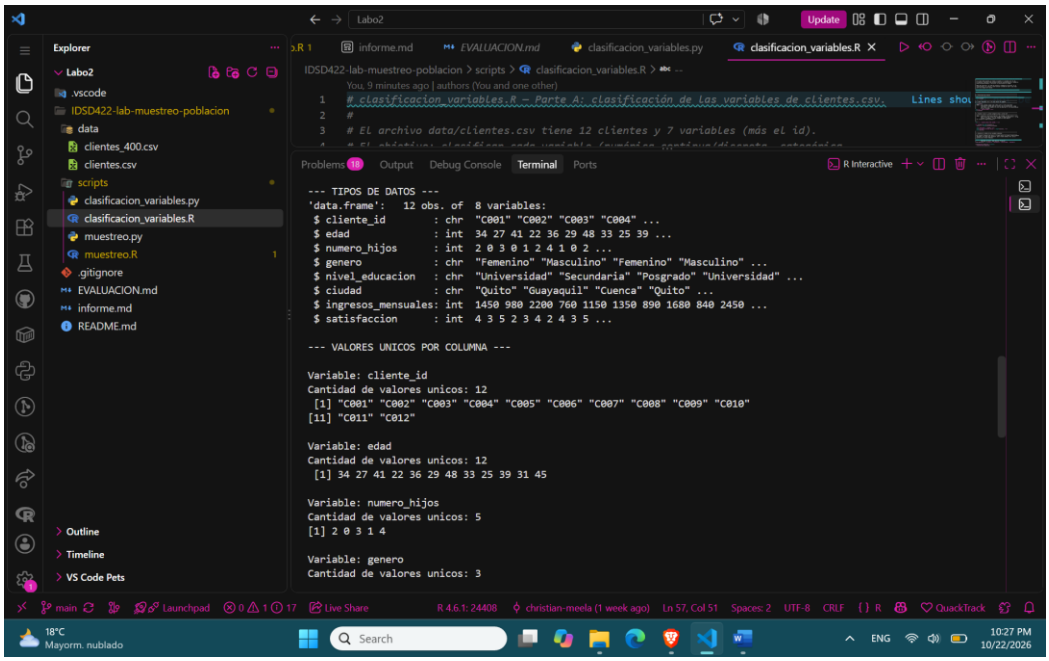


Figura 18. Ejecución de clasificación en R: estructura y valores únicos.